


```
[43]: print(me)

{'name': 'Samola', 'age': 24, 'height': 180.7, 'weight': 70, 'hair': 'black', 'eye': 'black', 'nationality': 'Nigerian', 'hair_color': 'black', 'eye_color': 'black'}

In [ ]: ##Adding a new key:value pair
me['hasDog'] = False
print(me)

In [ ]: me['nationality'] = me.get('nationality', 'Nigerian')
me

In [ ]: #Deleting a key
del me['hasDog']
print(me)
```

LOOPING THROUGH A DICTIONARY

With for loop, we can iterate through the keys and / or values of a dictionary. Let's see the various ways to achieve it.

```
In [ ]: for k in me:
        print(k)
```

As seen in the above code, only the key names were printed. When you loop through a dictionary, it only accesses its keys. But what if we need to access its values. How do we go about it?

```
In [ ]: for k in me:
        print(me[k])
```

Apart from using the key name within square brackets, the dictionary object has a method called values() which allows us to access all dictionary values.

```
In [ ]: me.values()

In [ ]: for v in me.values():
        print(v)
```

We have seen how to access a dictionary keys only or values only. But how do i know what each value represent? To access each key:value pair, we use the items() method. The items() method returns a list of tuples with each tuple containing the key and value pair.

```
In [ ]: me.items()

In [ ]: for key, value in me.items():
        print(f'{key} : {value}')
```

DICTIONARY OPERATORS

The dictionary object has two operators: **in** and **not in**. These operators check if the key exists in the dictionary or not.

```
In [ ]: print('height' in me)

In [ ]: print('year' in me)

In [ ]: print('height' not in me)

In [ ]: print('year' not in me)
```

DICTIONARY METHODS

In this lesson, we are going to use some of the commonly used dictionary methods. For more information on other methods, check [Dictionary methods](#)

```
In [ ]: #update the dictionary with key/value pairs. This method overwrites existing keys.
me.update({'dob':'20:3:1990', 'complexion': 'dark'})
print(me)

In [ ]: #Return an iterator over the keys of the dictionary
iterator = iter(me)
print(next(iterator))
print(next(iterator))

In [ ]: #return a copy of the dictionary
me_copy = me.copy()
print(me_copy)

In [ ]: #clear the dictionary
me_copy.clear()
print(me_copy)

In [ ]: #return the value of a key. set a default value if key does not exist. returns none if default is not set.
print(me.get('age', 0))
print(me.get('hasDog', True))

In [ ]: print(me.get('relationship'))
```

SWITCHING DATA STRUCTURES

Each data structure has different properties and characteristics. Sometimes we will want to change one data structure to another to take advantage of these differences. We can achieve this using the data structure's constructors.

```
In [ ]: ex_list = [2, 'apple', True, 3.5, 2]
ex_tuple = tuple(ex_list)
print(ex_tuple)

In [ ]: ex_set = set(ex_tuple)
print(ex_set)

In [ ]: ex_list = list(ex_set) #duplicates lost because of set
print(ex_list)
```

SORTING DATA STRUCTURES

Out of all the data structures we have studied so far, only list has a sort method. However, Python also has a sorted() function, which returns a sorted list of other data structures. By default, the sorted() function applied to a dictionary makes a list of the sorted dictionary keys. We must use the items() method if we want our output to be a list of the key-value pairs.

```
In [ ]: ex_list = list(map(str,ex_list))
#using the list sort() method
ex_list.sort()
print(ex_list) #uppercase letters comes first before lowercase

In [ ]: #using the sorted function
print(sorted(ex_list))

In [ ]: print(sorted(map(str,ex_set)))

In [ ]: print(sorted(map(str,ex_tuple)))

In [ ]: print(sorted(me))
print(sorted(me.items()))
```

ZIP() FUNCTION

The zip() function combines two or more sequences/iterators together into a list of tuples. If the sequences/iterators are of different length, the smallest length determines the length of the zip object. Zip() function can be very handy for creating a dictionary. Let's see how to do this using the me_list we created before. We are going to create another list to store key values matching each value in the me_list.

```
In [ ]: value_list = me_list
key_list = ['name', 'age', 'height', 'weight', 'hair', 'eye']

The zip() returns a zip object so we will need to convert it to a list. We can then convert the list of tuples to a dictionary using the dict() constructor.

In [ ]: key_value_pairs = list(zip(key_list, value_list))
print(key_value_pairs)

In [ ]: me_dict = dict(key_value_pairs)
print(me_dict)
```

EXERCISE

Write a program that prints the letters contained in a multiline string variable in decreasing order of frequency. Your program should convert all the input to lower case and only count the letters a-z. Your program should not count spaces, digits, punctuation, or anything other than the letters a-z.

```
In [ ]: 
```